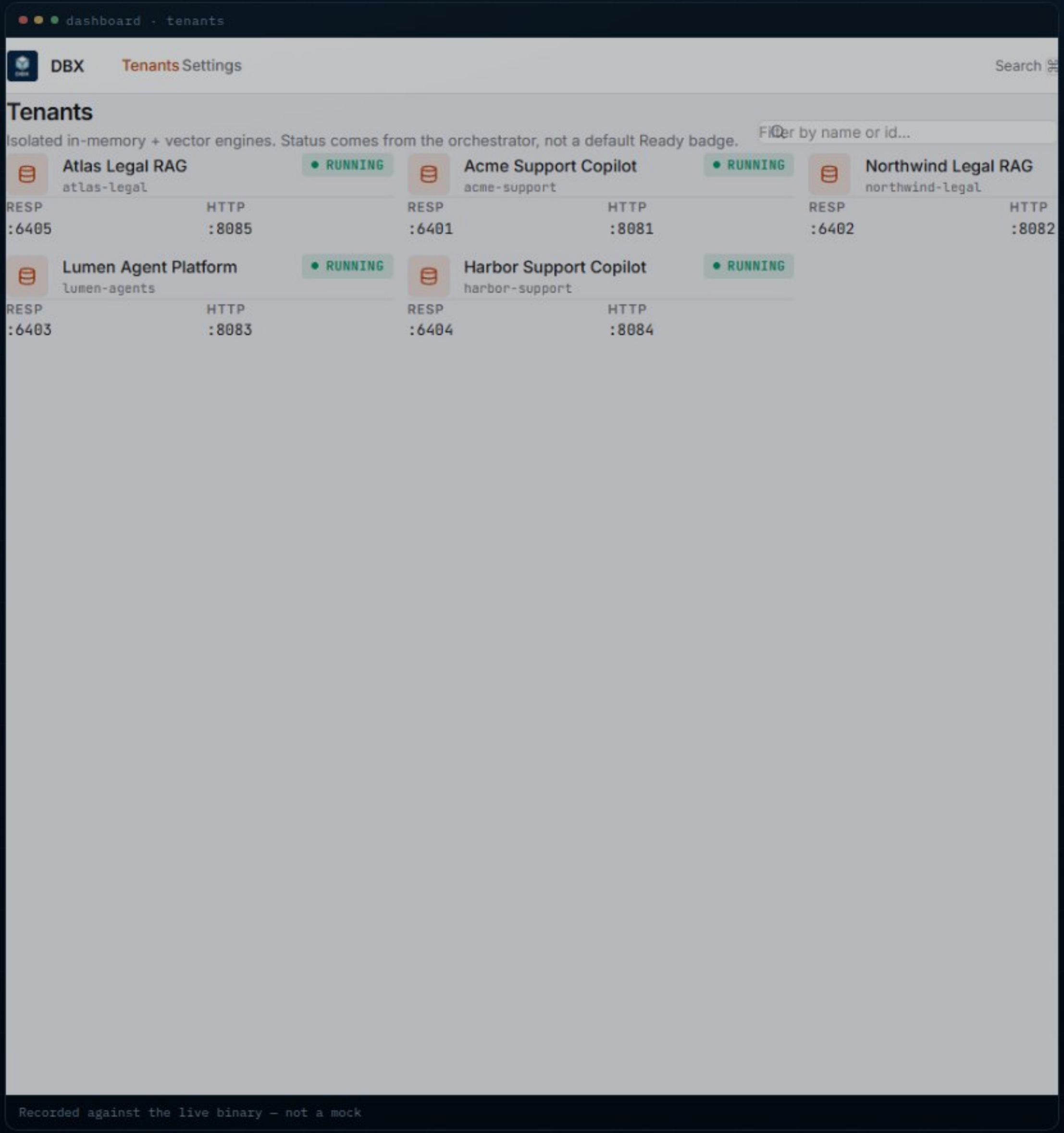
DATABASE EXTREME
PER-TENANT MEMORY ENGINE

INCUBATION BRIEFING · CONFIDENTIAL · SEPTEMBER 2026

The tenant is the database.

DBX is the per-tenant memory engine for AI products: one isolated store per customer, holding working state and vector memory. Not a shared cluster with a prefix.



Product **v1.1.0** Profile **100 tenants / node** License **BSL 1.1** Founder **Vansh Jain · DTU**

THESIS

One isolated engine per customer.

Every customer of an AI product needs private session state and private semantic recall. Today that is two systems plus a naming convention. DBX makes the tenant the unit of the database — directory, WAL, HNSW index, process, and key.

01

Working state

KV with TTL: session, scratchpad, rate counters. Same connection as recall.

02

Vector memory

SQ8 HNSW, mmap'd, sized for per-tenant working sets — not a billion-vector corpus.

03

Sealed domain

Linux `strict`: process, Landlock, envelope encryption, Unix `SO_PEERCRED`.

WHY NOW

Agents shipped. Isolation did not.

- AI agent platforms and vertical SaaS now give every end-customer a memory, not a chat transcript.
- Indirect prompt injection can force a similarity search that leaks neighbor context into the reasoning chain.
- Regulated buyers need “delete my data” as an API, not a sprint — GDPR/DPDP-shaped off-boarding.
- Running Redis *and* a vector DB doubles on-call, dual-writes, and bills.

Industry context (order of magnitude, definitions vary): vector databases ~$3.2B in 2025 → ~$8.95B by 2030 (MarketsandMarkets, 27.5% CAGR). Agentic applications of vector DBs ~$0.57B in 2026 → ~$1.73B by 2031 (Mordor Intelligence, 24.86% CAGR).

The architectural fork

Firecracker per user is a fleet of computers. Prefix filters are a hope. DBX is the missing third path: kernel-enforced tenant memory at ~14–17 MiB RSS per idle worker.

We do not claim the strongest sandbox. We claim the sealed unit is one customer’s KV plus vectors, with a key you can destroy.

PROBLEM

Shared clusters make tenancy your job.

WHAT TEAMS SHIP TODAY	WHAT IT COSTS THEM
Tenancy is a key prefix	One forgotten `tenant_id` is a cross-customer leak
Cache + vector DB	Dual writes, drift after crashes, two on-call surfaces
Backup is cluster-wide	Cannot restore or export a single customer
Deletion is a scan	“Delete my data” becomes an engineering project
Shared memory pool	A noisy tenant evicts a quiet tenant’s working set
Cost is guessed	No meter for what one customer actually consumes

None of this is fixed by making the shared cluster faster. It is a consequence of the unit of the database being the cluster.

SOLUTION

Isolation is a directory, a WAL, and a process.

CONTROL PLANE

HTTP :8000 · tenant lifecycle, scoped credentials, routing

RESP ingress :6380 · AUTH tenant:key_id secret, then route

TENANT A

KV + TTL

HNSW (SQ8, mmap)

Own WAL

Own snapshots

TENANT B

KV + TTL

HNSW (SQ8, mmap)

Own WAL

GET cannot open A's files

TENANT N

KV + TTL

HNSW (SQ8, mmap)

Own WAL

Purge shreds the wrap file

On-ramp, not a USP: existing RESP clients work. That lowers trial cost. It is not a claim that DBX replaces a tuned Redis cluster.

USPS 01–03

Five claims. Each is true in the binary.

USP 01

Tenant is a first-class object

`POST /api/provision` returns a live engine: directory, WAL, HNSW, snapshots, process. Backup, restore, hibernate, and purge operate on exactly one customer.

USP 02

State and memory in one engine

Session KV and semantic recall share a connection, a directory, and a backup zip. There is no second system to drift from.

Limit: periodic `.rdb` serializes KV; vectors live in mmap + WAL + seals.

USP 03

Cost scales with active tenants

SQ8 rows are ~4× smaller than float32. Idle tenants live in page cache. The metric we want to own is cost per active tenant, not signed-tenant count.

USPS 04–05

Self-hosted. Kernel-sealed on Linux.

USP 04

One binary, operator UI included

Embeddings never leave the customer's network. Dashboard, console, explorer, and vector playground compile into the orchestrator. No sidecar. No vendor to send data to.

USP 05

Isolation Kernel

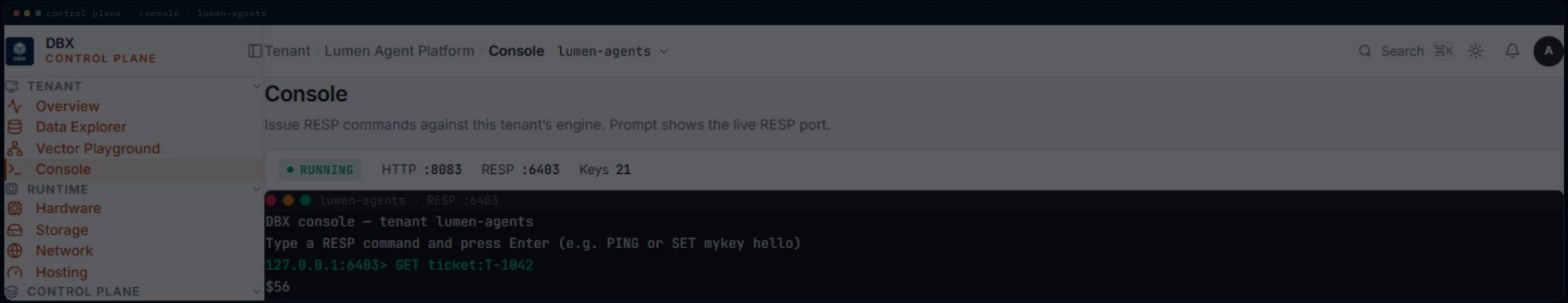
Linux `DBX_ISOLATION_MODE=strict`: dedicated `dbx-server`, Landlock LSM, cgroup v2 (best-effort in containers), AES-256-GCM over WAL / checkpoints / ids / HNSW, Unix sockets that only the orchestrator PID can open.

~15 MiB
Idle worker RSS (strict)

O(1)
Cryptographic shred of the DEK

0600
Unix sockets, peer-PID gated

LUKS
For SQ8 rows (mmap stays plaintext)



PRODUCT

The lifecycle is the product.

01
Provision
Isolated engine in milliseconds. Optional 0–2 async WAL replicas; primary still acks locally.

02
Keys
Reader / writer / tenant-admin. Reader cannot SET or VADD. Secret shown once. Revoke is immediate.

03
Query
Public data plane is RESP :6380. AUTH tenant:key. Python SDK: remember / recall / forget.

04
Backup
One-customer zip, SHA-256 manifest, restore with rollback. Hibernate stops the process, keeps the dir.

05
Shred
`purge=true` zeros the wrapped DEK, then deletes the directory. Blast radius: one tenant.

Certified v1 surface: durable strings + vectors. Raft, cluster/sharding, and non-string RESP mutation families fail closed.

WHO BUYS

Teams whose customers each need memory.

Ideal customer

- AI agent platforms — private recall per end-customer
- Vertical AI SaaS — legal, medical, support, finance corpora
- On-prem / regulated — one binary inside the customer network
- Teams hand-rolling tenancy over a cache and a vector store

We say no quickly

One tenant, one workload. Billion-vector ANN. SQL, joins, or a system of record. A fully managed zero-ops Pinecone-style service as the reason to buy.

DBX sits in front of the system of record and underneath the agent. That is the whole territory.

WHITE SPACE

Giants optimize the cluster. We sell a different unit.

IF YOU NEED	RIGHT TOOL	WHY NOT DBX
Peak single-instance KV	Redis / Dragonfly	Their design target; our cycles go to isolation
Billion-vector ANN + sharding	Qdrant / Milvus	Our indexes are per-tenant working sets
Fully managed vectors	Pinecone	We are software you run — deliberately
Joins, SQL, system of record	Postgres + pgvector	DBX is memory, not the ledger

Per-tenant isolation is not a row on those comparison charts. Framed as “Redis alternative,” our best property looks like a quirk. We do not take that fight.

STATUS · V1.1.0

Shipped. Certified. Open.

GO
Single-node production profile

100 / 100k
Tenants per node · vectors per tenant

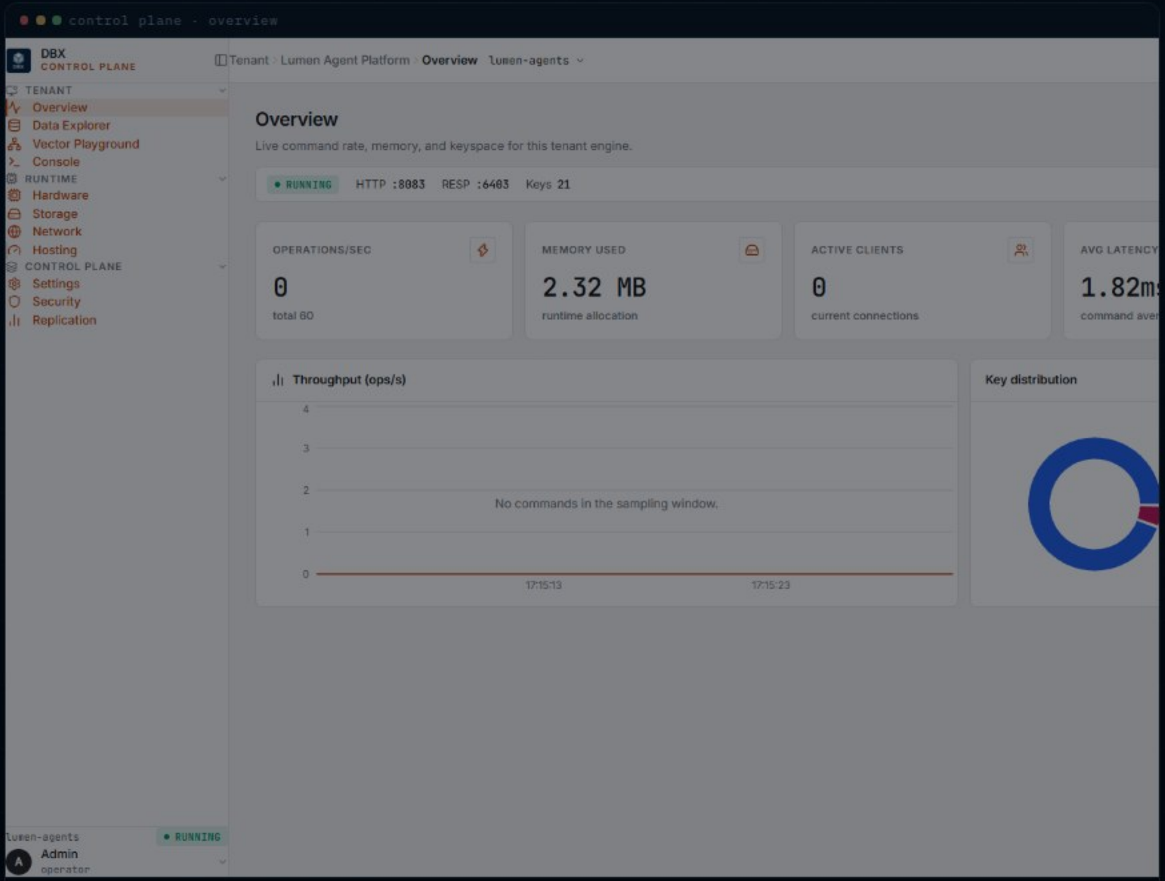
v1.1.0
Isolation Kernel · Unix data plane

BSL 1.1
Free in your SaaS · paid if you sell ours

What exists today

- Single RESP ingress on :6380, scoped keys, quotas, usage API
- Linux strict Isolation Kernel + GHCR orchestrator image on release
- Python SDK, LangChain and Next.js examples, embedded dashboard
- Async WAL replicas; data-plane Raft still fail-closed
- Public site, 5m18s walkthrough, DEV.to Isolation Kernel article kit

We do not claim ARR, customer logos, SOC 2, a cluster, or a managed cloud. Windows is the certification host; Linux is the production USP. `hello@dbxdb.io` has no MX yet.



Shipped operator UI, compiled into the orchestrator

PROOF · ONE NODE · ONE TENANT · 27 AUG 2026

Isolation is not expensive.

186k
SET ops/s · pipeline 64 · WAL everysec

285k
GET ops/s · same run

7.2k
Vector ingest /s · 100k × 128, 8 shards

2.3 ms
ANN p50 · p95 3.1 · p99 3.7 ms

Recall@10

Mean 0.920 · p05 0.800 · SQ8 vs float32 brute force. Quantization has a quality cost; we publish it.

Strict density

Idle sealed worker ~14–17 MiB RSS. 100 sealed tenants ≈ 1.5 GiB process RSS before corpus. Use inprocess when density is the goal.

Noisy neighbor

Quota rejection isolates OOM. Quiet GET stayed within the isolation harness. Neighbours keep serving.

Not a competitor ranking. Measure on your hardware. Linux CI re-runs the vector harness.

MARKET

A category the cluster vendors do not serve.

TAM

Agent memory + vector infra

Vector DB market ~\$3.2B (2025) → ~\$8.95B (2030), 27.5% CAGR (MarketsandMarkets). Agent memory infrastructure ~\$1.62B now → ~\$15.2B by 2034, 28.3% CAGR (HTF).

SAM

Multi-tenant agent memory

Agentic AI applications in vector databases ~\$0.57B (2026) → ~\$1.73B (2031), 24.86% CAGR (Mordor). Self-hosted, per-customer isolation is the slice those reports do not name.

SOM · 24 months

Design-partner wedge

Operational target, not a revenue forecast: 10 vertical-AI SaaS design partners; 3 commercial-license conversations for managed-DBX use. No invented ARR.

Research houses disagree on definitions. We cite ranges as order of magnitude. DBX wins if buyers bill per customer and cannot tolerate prefix tenancy — not if we out-benchmark Redis.

BUSINESS MODEL

Free in your product. Paid when you sell ours.

Self-host

Free · BSL 1.1

Production, personal, or commercial. Embed inside your own SaaS — each of your customers is a tenant. Converts to Apache 2.0 after four years.

Commercial

Talk · managed DBX

Required if DBX itself is the service you sell to third parties. Not a gray area in the additional use grant. The billable unit is the tenant.

Support

Talk · not a public SKU

Named contacts, SLA language for procurement. No fake monthly tiers on the website. Price follows workload shape.

Why this converts: the people we most want using DBX are companies embedding it. The license permits that. Cash arrives when someone wants to offer DBX as infrastructure — or when we elect to run the managed service the BSL reserved for us.

Land in the binary. Expand on the tenant.

Motion

- Developer-led: GitHub, RESP clients, 15-minute Python path, Isolation Kernel article
- Vertical design partners: legal, support, on-prem AI SaaS with a data-residency story
- Incubator network: first 10 design partners, not a paid ads plan
- Trust path: live MX, commercial contracting, SOC 2 readiness (not a badge we have)

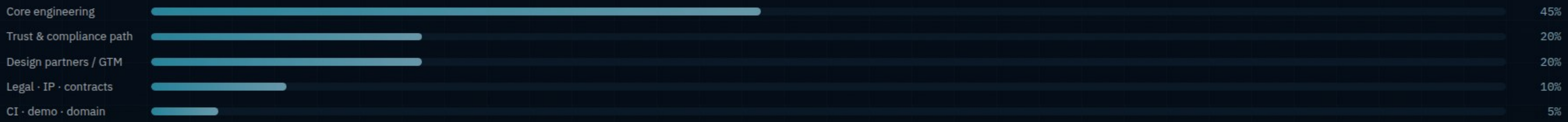
Product sequencing

- Now: v1.1.0 single-node, Isolation Kernel, usage meter
- Next: published Linux density, engine coverage, promote-without-restart
- Then: larger soak beyond 100 tenants/node when measured
- Company decision: managed DBX when design partners pull, not before

THE ASK

Incubation to first commercial licenses.

We are raising an 18-month operating partnership — not a priced round invented for this slide. The envelope scales; the mix does not.



Engineering

Density beyond 100/node, replica promote, engine coverage, optional float32 accuracy mode.

Trust

Register dbxdb.io + MX, commercial license paper, SOC 2 readiness program, LUKS operator path.

Gates we accept

10 design partners · 3 commercial conversations · Linux density published · mailbox live.

TEAM

Founder-built systems software.

Founder

Vansh Jain

Engineering undergraduate, Delhi Technological University. Designed and shipped DBX: Go 1.25 engine, Isolation Kernel (Landlock, envelope encryption, `SO_PEERCRED`), SQ8 HNSW, orchestrator, dashboard, Python SDK, and the public site.

Public identity: github.com/vanshjain-0702 · License: DBX Authors

What incubation adds

- Legal: commercial license templates, IP hygiene, contracting
- GTM: first design-partner introductions in vertical AI SaaS
- Trust: domain, mailbox, security-review process, compliance coaching
- Not required: a rewrite of the kernel. The product is the proof of work.

RISKS WE ALREADY NAME

Diligence, not a badge wall.

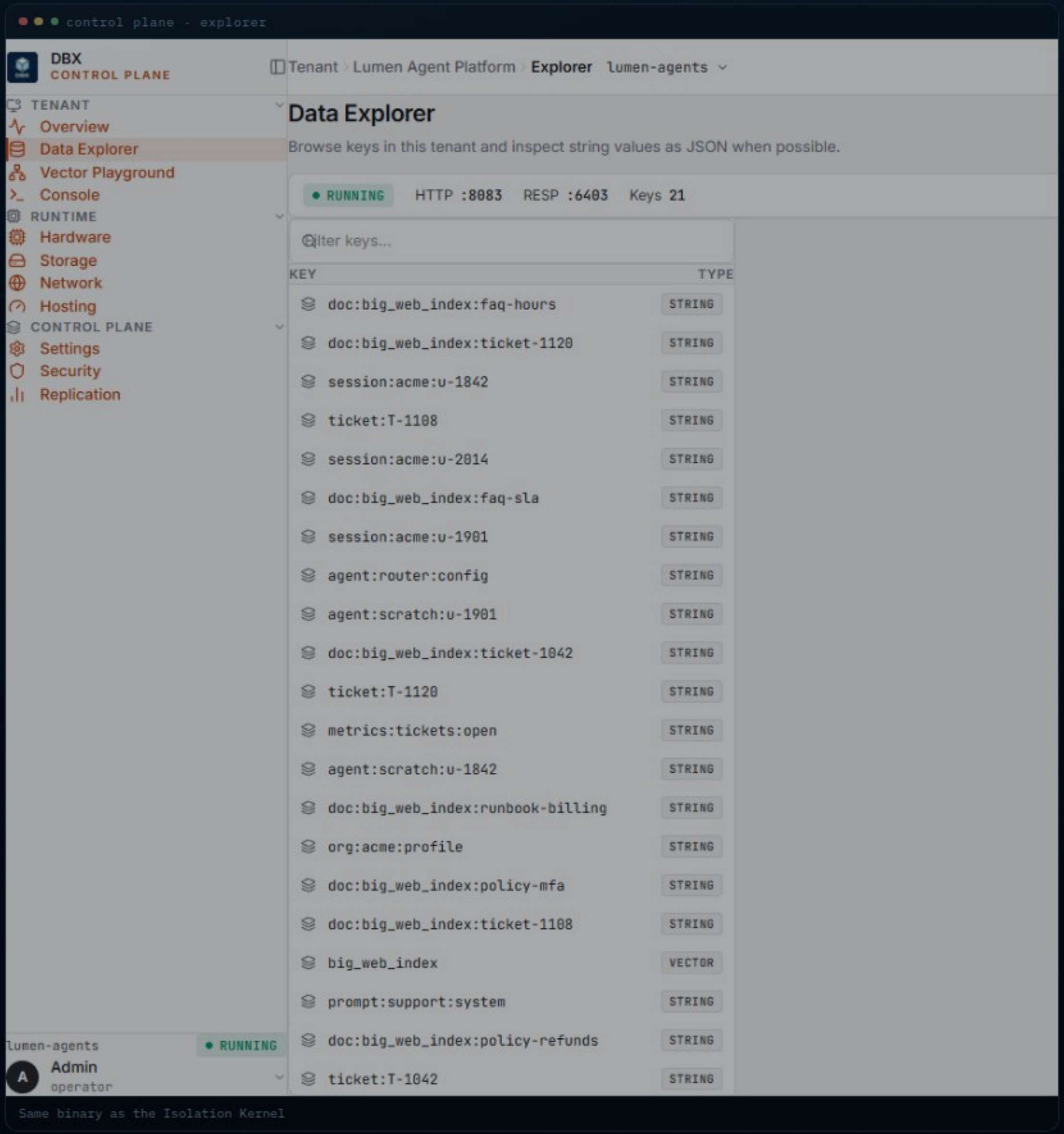
- SQ8 `.vec` rows stay mmap'd and unencrypted so idle tenants stay cheap — use LUKS/fscrypt.
- Landlock governs file opens, not `connect()` or `stat()`. Sockets are `SO_PEERCRED` + mode 0600.
- No network restriction on workers. Data in use is plaintext in the process.
- cgroup limits need host delegation; most containers log and continue.
- Certified cap remains 100 tenants/node until a larger soak is published.
- Single-node profile. Cluster/Raft on the data plane fail closed.
- Not SOC 2. Not a managed cloud. Not a Redis replacement.
- Solo founder concentration — incubation must add GTM and legal, not deny it.
- Pre-revenue: conversion depends on design partners, not a forecast spreadsheet.
- Domain MX not live; GitHub is the channel that currently delivers.

DATABASE EXTREME
PER-TENANT MEMORY ENGINE

CLOSE

Give every customer a **database**, not a prefix.

We are asking incubation to take a shipped Isolation Kernel to the first ten design partners and the first commercial licenses — with the same honesty we use in the code.



Live deck **pitch.html** Code **github.com/vanshjain-0702/DBX-Database-Extreme** Docs **github.io/DBX-Database-Extreme**

Walkthrough **demo.html**

APPENDIX · NON-GOALS

Coherence is a feature.

OFF THE ROADMAP	WHY
Beating Redis on single-instance KV	Wrong buyer, unwinnable axis
Sharded billion-vector ANN	Wrong index size for our thesis
Managed DBX cloud as a default	BSL exists so this stays a company decision
SQL / joins / cross-entity ACID	We are memory, not the system of record
OLAP over history	Wrong shape entirely

Source of truth: docs/positioning.md · Isolation Kernel: docs/isolation.md · License: LICENSE (BSL 1.1 → Apache 2.0 after four years).